

Name: Ravi Shankar Tripathi

Github Link: https://github.com/RAshtr/Flask_and_MongoDB_Assignment

Source Code and output screenshots:

Task 1: JSON API Route:

[app.py](#):

```
import json
from flask import Flask, jsonify, render_template, request, redirect, url_for
from pymongo import MongoClient
from pymongo.errors import ConnectionFailure, PyMongoError

app = Flask(__name__)

# Yahan apna actual MongoDB Atlas connection string daalna hai:
MONGO_URI =
"mongodb+srv://ravitripathi878_db_user:P5VezJSn0Er0fxKt@cluster0.hxbjivs.m
ongodb.net/?appName=Cluster0"

try:
    client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=3000)
    db = client["devops_assignment_db"]
    collection = db["submissions"]
except Exception as e:
    client = None

@app.route('/api', methods=['GET'])
def get_api_data():
    try:
        with open('data.json', 'r') as file:
            data = json.load(file)
        return jsonify(data), 200
    except Exception as e:
        return jsonify({"error": "Failed to read backend data file",
"details": str(e)}), 500
```

```

@app.route('/', methods=['GET', 'POST'])
def handle_form():
    if request.method == 'POST':
        name = request.form.get('name', '').strip()
        email = request.form.get('email', '').strip()
        course = request.form.get('course', '').strip()

        if not name or not email or not course:
            return render_template('form.html', error="All fields are
required. Please fill out the full form.")

        try:
            if client is None:
                raise ConnectionFailure("Could not connect to MongoDB
Atlas.")

            client.admin.command('ping')

            submission_data = {
                "name": name,
                "email": email,
                "course": course
            }
            collection.insert_one(submission_data)
            return redirect(url_for('success_page'))

        except Exception as err:
            return render_template('form.html', error=f"Database Error:
{str(err)}")

        return render_template('form.html', error=None)

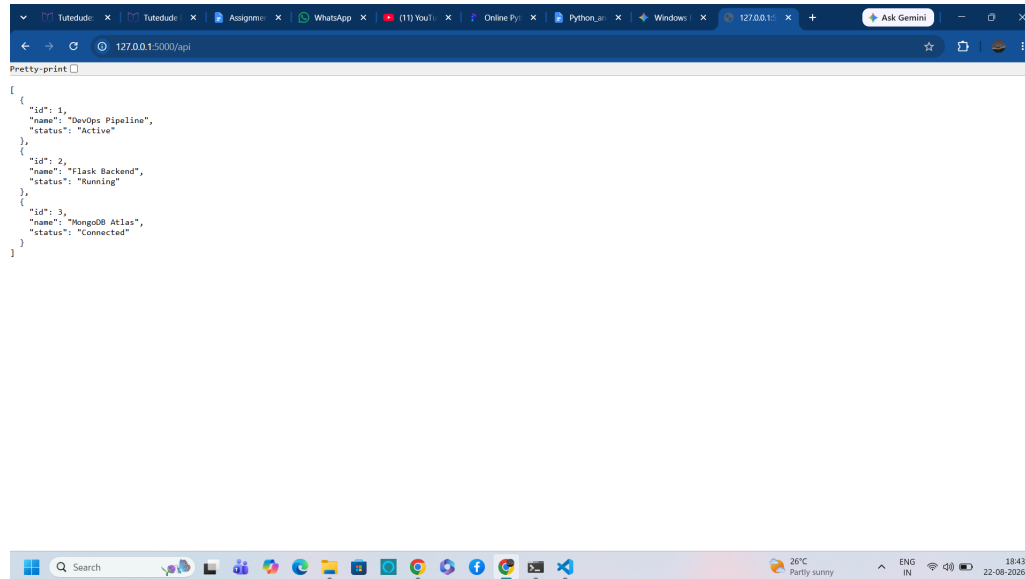
@app.route('/success')
def success_page():
    return render_template('success.html')

if __name__ == '__main__':
    app.run(debug=True, port=5000)

```

Data.json:

```
[
  {"id": 1, "name": "DevOps Pipeline", "status": "Active"},
  {"id": 2, "name": "Flask Backend", "status": "Running"},
  {"id": 3, "name": "MongoDB Atlas", "status": "Connected"}
]
```



Technical Explanation:

1. **Data Storage (data.json):** A backend file stores static records in JSON format representing system and pipeline statuses.
2. **Endpoint Implementation (@app.route('/api')):**
 - When accessed via standard **GET** request, the route opens and reads **data.json** using Python's built-in **json.load()** function.
 - The data is serialized into JSON standard output using Flask's native **jsonify()** utility.
 - Returns an **HTTP 200 OK** status on success and includes error handling returning an **HTTP 500 Internal Server Error** if file I/O operations fail.

Task 2: Frontend Form with MongoDB Atlas:

Form.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DevOps Assignment - Flask & MongoDB</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 40px;
background-color: #f4f6f8; }
    .card { background: white; padding: 25px; border-radius: 8px;
max-width: 450px; box-shadow: 0 2px 4px rgba(0,0,0,0.1); }
    .error { color: #d9534f; background: #fdf7f7; border: 1px solid
#d9534f; padding: 10px; border-radius: 4px; margin-bottom: 15px; }
    input[type="text"], input[type="email"] { width: 100%; padding:
8px; margin: 8px 0 16px; border: 1px solid #ccc; border-radius: 4px;
box-sizing: border-box; }
    input[type="submit"] { background-color: #007bff; color: white;
border: none; padding: 10px 15px; border-radius: 4px; cursor: pointer;
width: 100%; font-size: 16px; }
  </style>
</head>
<body>
  <div class="card">
    <h2>Submit Details</h2>
    {% if error %}
      <div class="error">{{ error }}</div>
    {% endif %}
    <form action="/" method="POST">
      <label for="name">Name:</label>
      <input type="text" id="name" name="name" placeholder="Enter
your full name">
```

```

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" placeholder="Enter
your email">

        <label for="course">Course/Role:</label>
        <input type="text" id="course" name="course"
placeholder="e.g., DevOps Engineer">

        <input type="submit" value="Submit to MongoDB">
    </form>
    <p style="margin-top: 15px;"><a href="/api" target="_blank">View
/api JSON Route</a></p>
</div>
</body>
</html>

```

Success.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Success</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px;
background-color: #f4f6f8; }
        .card { background: white; padding: 25px; border-radius: 8px;
max-width: 450px; box-shadow: 0 2px 4px rgba(0,0,0,0.1); text-align:
center; }
        .success { color: #28a745; font-size: 20px; font-weight: bold; }
        a { display: inline-block; margin-top: 20px; color: #007bff;
text-decoration: none; }
    </style>
</head>
<body>
    <div class="card">
        <p class="success">Data submitted successfully</p>
        <p>Your record has been stored in MongoDB Atlas.</p>
        <a href="/">Submit another response</a>
    </div>
</body>

```

```
</html>
```

Technical Explanation:

1. Cloud Database Connectivity:

- Utilizes the `pymongo` driver with SRV support (`pymongo[srv]`) to establish an encrypted, remote connection with MongoDB Atlas.
- Initializes client connectivity with a defined server selection timeout to handle network resilience gracefully.

2. Server-Side Validation & Error Handling (Without Redirection):

- The `/` route listens to both `GET` and `POST` requests.
- When a form is submitted, the server sanitizes and checks input fields (`name`, `email`, `course`).
- **Error State:** If any field is empty or if a database connection error occurs, the server avoids redirection and re-renders `templates/form.html` passing the error message directly into the template context (`render_template('form.html', error=...)`).

3. Database Insertion & Success Redirection:

- **Success State:** Upon successful validation, the record is encapsulated into a Python dictionary and inserted into the `submissions` collection using `collection.insert_one()`.
- On confirmation of insertion, the user is redirected via `redirect(url_for('success_page'))` to the dedicated `/success` endpoint (`templates/success.html`), which displays the required confirmation: *"Data submitted successfully"*.

127.0.0.1:5000

Submit Details

All fields are required. Please fill out the full form.

Name:

Email:

Course/Role:

[Submit to MongoDB](#)

[View /api JSON Route](#)

127.0.0.1:5000/success

Data submitted successfully

Your record has been stored in MongoDB Atlas.

[Submit another response](#)

Cluster0 Data

Success

Ask Gemini

cloud.mongodb.com/v2/6a89a70230b5f5419c7b64a6#/explorer/6a89a99fe5fb2d113ca83bf2/devops_assignment_db/submissions/find

ORGANIZATION
Ravi's Org - 2026-08-...

PROJECT
Project 0

Compare tiers

Data Explorer

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

- admin
- devops_assignment_db
 - submissions
- local
- sample_mflix

submissions

Cluster0 > devops_assignment_db > submissions

Documents 1

Aggregations

Schema

Indexes 1

Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

BULK

EXPORT CODE

25 1-1 of 1

View monitoring

Visualize Your Data

```
{
  "_id": ObjectId('6a89adbe75a9dc9b81efdcd6'),
  "name": "Ravi Shankar Tripathi",
  "email": "ravitripathi878@gmail.com",
  "course": "DevOps Engineer"
}
```

Alert System Status: All Good

©2026 MongoDB, Inc. [Status](#) [Terms](#) [Privacy](#) [Atlas Blog](#) [Contact Sales](#)

Search

26°C
Partly cloudy

ENG
IN